

CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 2 Report

Team Members: **Connor Moroney**
 Owen Gilbertson

Project Teams Group: **D07**

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[1.a] Come up with a global list of the datapath values and control signals that are required during each pipeline stage.

Incoming signals that are needed for each stage:

IF:

Datapath – **s_NextInstAddr, s_Inst**

Control – **s_PC_SEL, s_PC_WE, iInstLd**

ID:

Datapath - **s_Inst, s_rs1_data, s_rs2_data, s_rs1, s_rs2, s_imm12bit, s_imm20bit, s_DecImmType**

Control - **None**

EX:

Datapath - **s_rs1_data, s_rs2_data, s_imm**

Control - **s_ALUOp, s_ALUSrc, s_signed, s_Branch, s_func3, s_Jump, s_auiopc, s_upperIMM**

MEM:

Datapath - **s_DMemAddr, s_DMemData, s_DMemOut, s_LoadData**

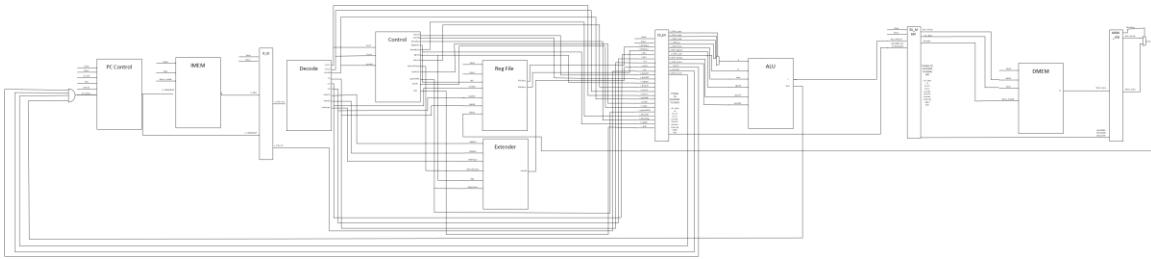
Control - **s_DMemWr, s_func3**

WB:

Datapath - **s_ALUResult, s_LoadData, s_RegWrAddr, s_RegWrData**

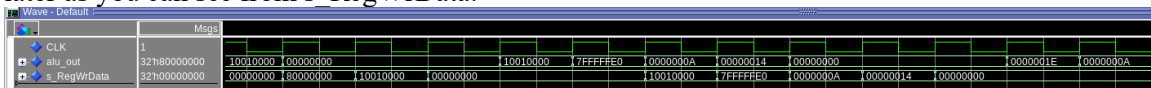
Control - **s_RegWr, s_MemReg**

[1.b.ii] high-level schematic drawing of the interconnection between components.



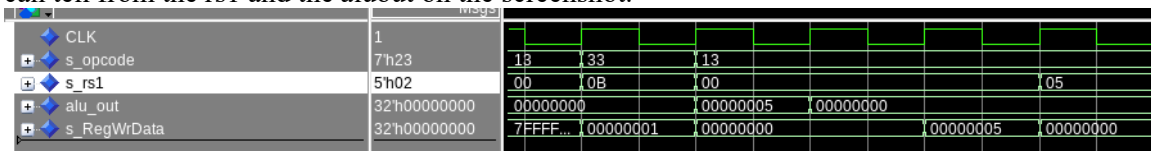
[1.c.i] include an annotated waveform in your writeup and provide a short discussion of result correctness.

The following image shows the results of the Proj2_base_test.s including the nops to be suitable for the software pipeline. As you can see, the 0's in the alu show the nops in a few different places here on this specific screenshot. However, you can see it still writes correctly a few cycles later as you can see from s_RegWrData.

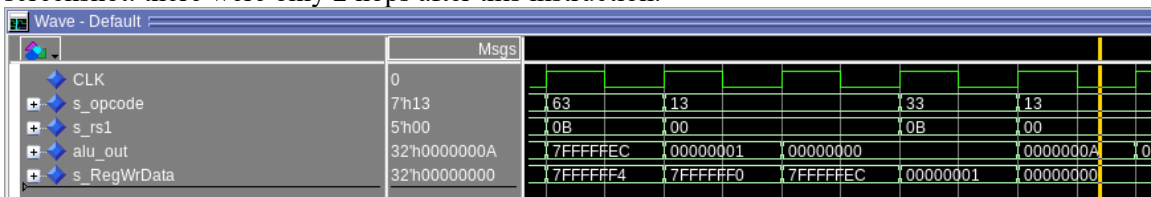


[1.c.ii] Include an annotated waveform in your writeup of two iterations or recursions of these programs executing correctly and provide a short discussion of result correctness. In your waveform and annotation, provide 3 different examples (at least one data-flow and one control-flow) of where you did not have to use the maximum number of NOPs.

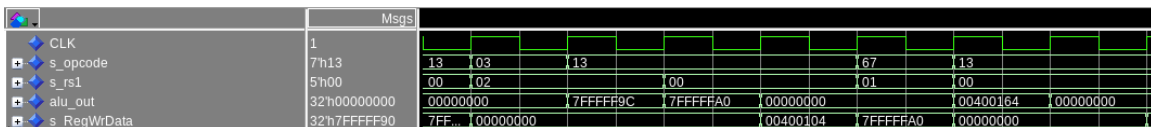
Data Flow: used an R-type instruction, which had only 3 nops after this instruction, which you can tell from the rs1 and the aluout on the screenshot.



Control Flow: Branch type instruction based on the opcode: 63. As you can see from the screenshot: there were only 2 nops after this instruction.

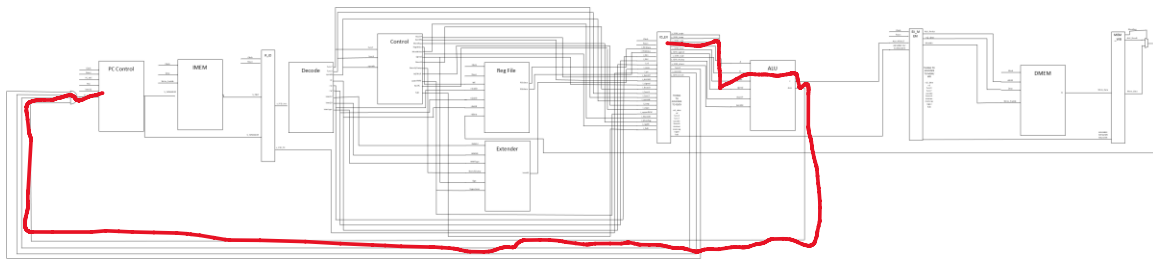


Data Flow: Load type instruction: needed 2 nops after this instruction: which is displayed again from the screenshot below:



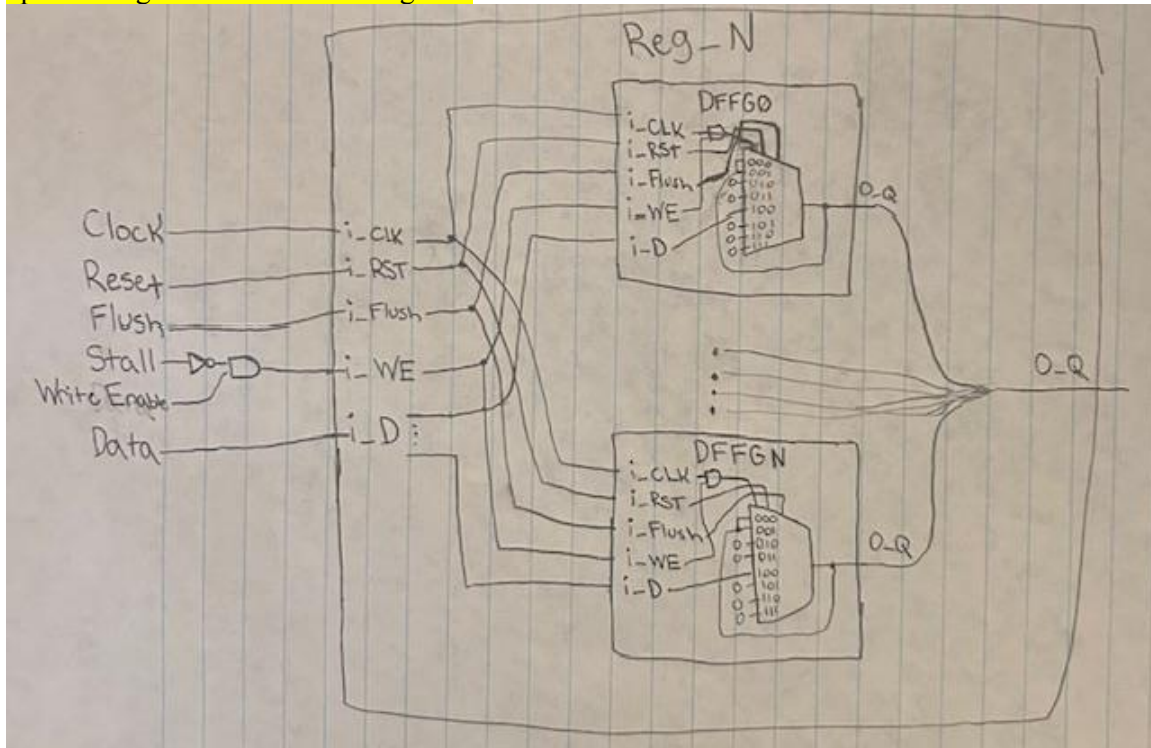
[1.d] report the maximum frequency your software-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

Fmax = 49.29 MHz



ID_EX -> ALU -> PC Control

[2.a.ii] Draw a simple schematic showing how you could implement stalling and flushing operations given an ideal N-bit register.



[2.a.iii] Create a testbench that instantiates all four of the registers in a single design. Show that values that are stored in the initial IF/ID register are available as expected four cycles later, and that new values can be inserted into the pipeline every single cycle. Most importantly, this testbench should also test that each pipeline register can be individually stalled or flushed.



As you can see, the data stored in a pipeline register gets passed to the next pipeline register. Stalling makes it so that the register doesn't update its value. Flushing gets rid of the value in the register (sets to 0).

[2.b.i] list which instructions produce values, and what signals (i.e., bus names) in the pipeline these correspond to.

- **ADD/SUB/AND/OR/XOR/SLT/SLTU/SLL/SRL/SRA**
 - s_ALUResult, s_EXMEM_ALU_result, s_MEMWB_ALU_result, s_RegWrData
- **ADDI/ANDI/ORI/XORI/SLTI/SLTIU/SLLI/SRLI/SRAI**
 - s_ALUResult, s_EXMEM_ALU_result, s_MEMWB_ALU_result, s_RegWrData
- **LUI**
 - s_ALUResult, s_EXMEM_ALU_result, s_MEMWB_ALU_result, s_RegWrData
- **AUIPC**
 - s_Op1, s_ALUResult, s_EXMEM_ALU_result, s_MEMWB_ALU_result, s_RegWrData
- **JAL/JALR**
 - s_Op1, s_ALUResult, s_EXMEM_ALU_result, s_MEMWB_ALU_result, s_RegWrData
- **LW/LB/LH/LBU/LHU**
 - s_DMemOut, s_LoadData, s_MEMWB_MEM_data, s_RegWrData
- **SW**
 - Data: s_ALU_Op2, s_EXMEM_rs2_data, s_DMemData
 - Address: s_ALUResult, s_EXMEM_ALU_result, s_DMemAddr

[2.b.ii] List which of these same instructions consume values, and what signals in the pipeline these correspond to.

- **ADD/SUB/AND/OR/XOR/SLT/SLTU/SLL/SRL/SRA**
 - s_rs1_data, s_IDEX_rs1_data, s_ALU_Op1
 - s_rs2_data, s_IDEX_rs2_data, s_ALU_Op2
- **ADDI/ANDI/ORI/XORI/SLTI/SLTIU/SLLI/SRLI/SRAI**
 - s_rs1_data, s_IDEX_rs1_data, s_ALU_Op1
 - s_IDEX_imm
- **LUI**
 - s_IDEX_imm
- **AUIPC**
 - s_IDEX_imm
 - s_IDEX_pc

- **JAL**
 - s_IDEX_imm
 - s_IDEX_pc
- **JALR**
 - s_rs1_data, s_IDEX_rs1_data, s_ALU_Op1
 - s_IDEX_imm
- **LW/LB/LH/LBU/LHU**
 - s_rs1_data, s_IDEX_rs1_data, s_ALU_Op1
 - s_IDEX_imm
- **SW**
 - s_rs1_data, s_IDEX_rs1_data, s_ALU_Op1
 - s_rs2_data, s_IDEX_rs2_data, s_ALU_Op2
 - s_IDEX_imm
- **BEQ/BNE/BLT/BGE/BLTU/BGEU**
 - s_rs1_data, s_IDEX_rs1_data, s_ALU_Op1
 - s_rs2_data, s_IDEX_rs2_data, s_ALU_Op2
 - s_IDEX_imm

[2.b.iii] generalized list of potential data dependencies. From this generalized list, select those dependencies that can be forwarded (write down the corresponding pipeline stages that will be forwarding and receiving the data), and those dependencies that will require hazard stalls.

Potential Data Dependencies:

- **Compute-Use**
 - **Forward: EX/MEM => EX** (Old ALU output used for new ALU input)
 - **OR if Hazard is in MEM Stage:**
 - **Forward: MEM/WB => EX** (Old ALU output used for new ALU input)
- **Load-Use**
 - **STALL** until loaded from memory (wait until MEM stage is finished)
 - **Forward: MEM/WB => EX** (Mem output used for ALU input)
- **Writeback-Use**
 - **Forward: WB => ID** (Update RegFile to allow same write and read)

[2.b.iv] global list of the datapath values and control signals that are required during each pipeline stage

IF:

Datapath – s_NextInstAddr, s_Inst

Control – s_PC_SEL, s_PC_WE, iInstLd

ID:

Datapath – **s_Inst, s_rs1_data, s_rs2_data, s_rs1, s_rs2, s_imm12bit, s_imm20bit, s_DecImmType**

Control – **None**

EX:

Datapath – **s_IDEX_rs1_data, s_IDEX_rs2_data, s_IDEX_imm, s_IDEX_pc, s_Op1**

Control – **s_IDEX_ALUOp, s_IDEX_ALUSrc, s_IDEX_signed, s_IDEX_Branch, s_IDEX_func3, s_IDEX_func7, s_IDEX_opcode, s_IDEX_Jump, s_IDEX_auipc, s_IDEX_upperIMM, s_IDEX_MemWr, s_IDEX_MemReg, s_IDEX_RegWr, s_IDEX_Halt**

MEM:

Datapath – **s_EXMEM_ALU_result, s_EXMEM_rs2_data, s_DMemAddr, s_DMemData, s_DMemOut, s_LoadData**

Control – **s_EXMEM_MemWr, s_EXMEM_func3, s_EXMEM_MemReg**

WB:

Datapath – **s_MEMWB_ALU_result, s_MEMWB_MEM_data, s_MEMWB_rd, s_RegWrData**

Control – **s_MEMWB_RegWr, s_MEMWB_MemReg, s_MEMWB_Halt**

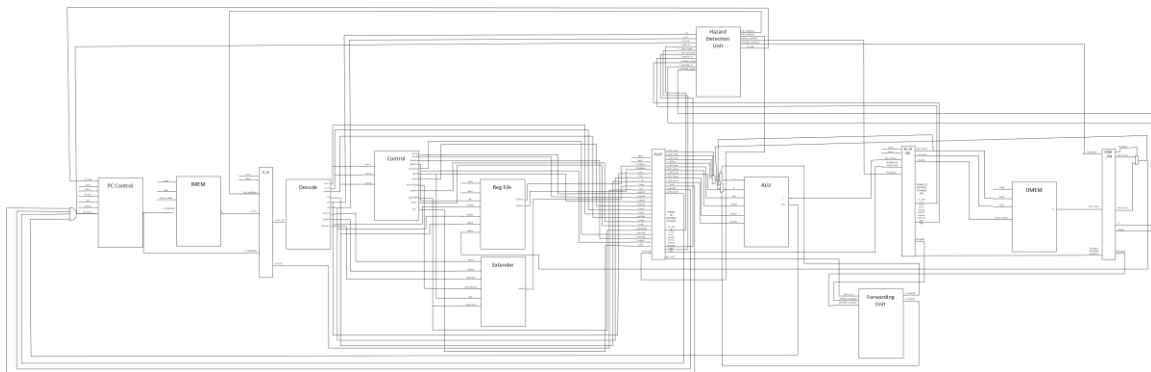
[2.c.i] list all instructions that may result in a non-sequential PC update and in which pipeline stage that update occurs.

- **JAL/JALR**
 - EX
- **BEQ/BNE/BLT/BGE/BLTU/BGEU**
 - EX

[2.c.ii] For these instructions, list which stages need to be stalled and which stages need to be squashed/flushed relative to the stage each of these instructions is in.

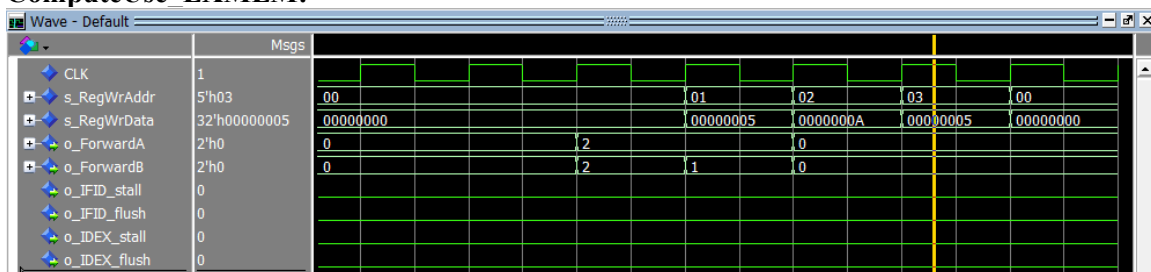
- **JAL/JALR**
 - STALL: NONE
 - FLUSH: IF/ID and ID/EX when jump happens
- **BEQ/BNE/BLT/BGE/BLTU/BGEU**
 - STALL: NONE
 - FLUSH: IF/ID and ID/EX only if branch is taken

[2.d] implement the hardware-scheduled pipeline using only structural VHDL. As with the previous processors that you have implemented, start with a high-level schematic drawing of the interconnection between components.



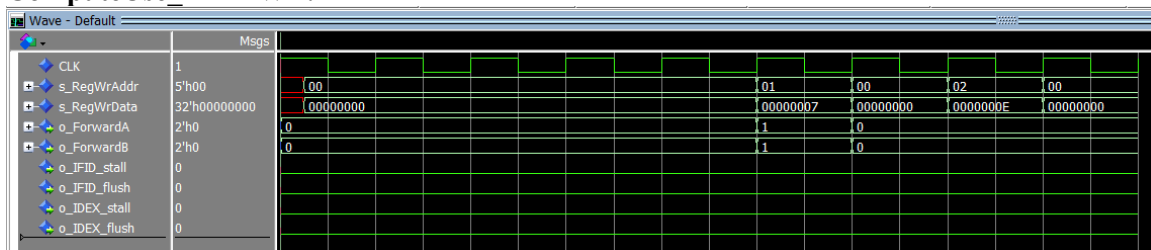
[2.e – i, ii, and iii] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

ComputeUse_EXMEM:



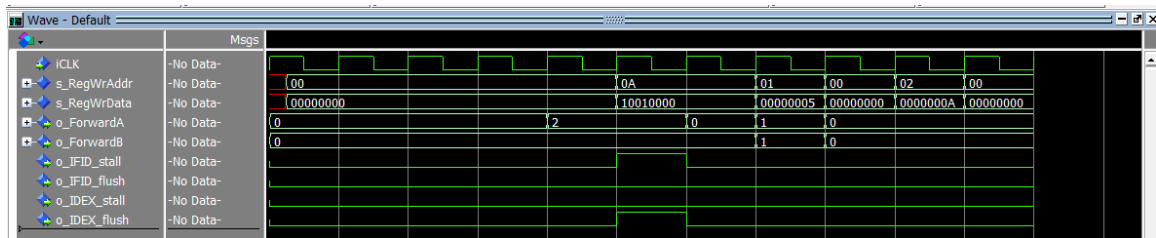
Multiple Forward paths are used here; No stalls or flushes are needed.

ComputeUse_MEMWB:



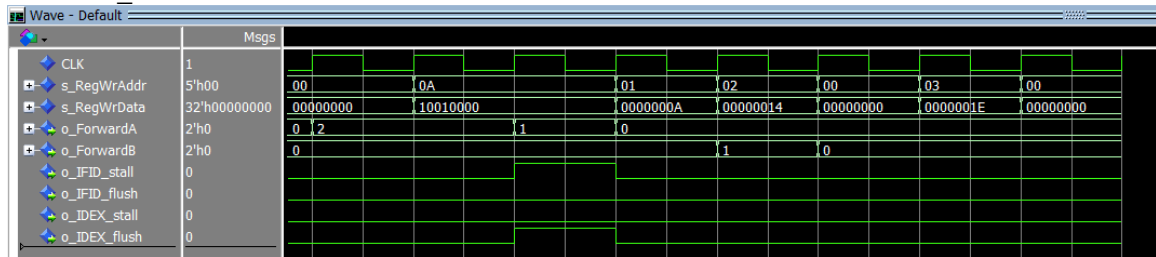
Again, multiple Forward paths are used here; No stalls or flushes are needed.

LoadUse_1:



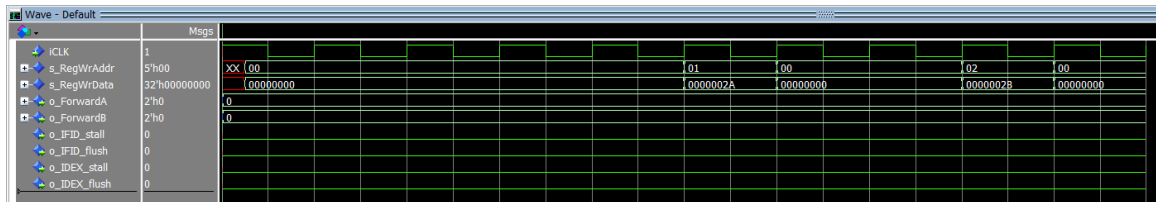
For the Load-Use hazard, a stall must be used until the value is loaded from memory and can be forwarded.

LoadUse_Double:



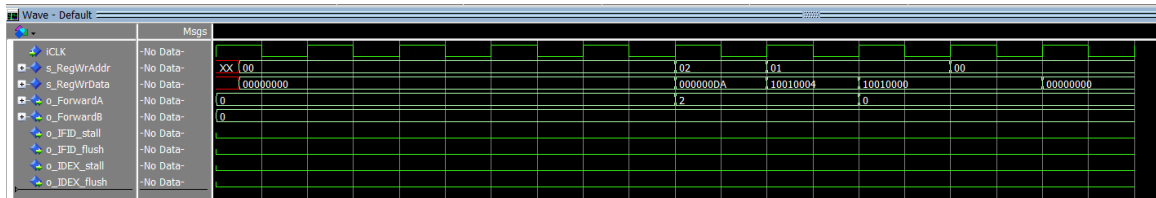
Similar result as above. Stall needed and multiple forwards

WBUse:



No explicit forwards are needed for WB-Use hazards. But the RegFile was updated to read the value being written if rs1 or rs2 matches the destination register.

StoreAddrFwd:



No stalls or flushes are needed, but a forward is used.

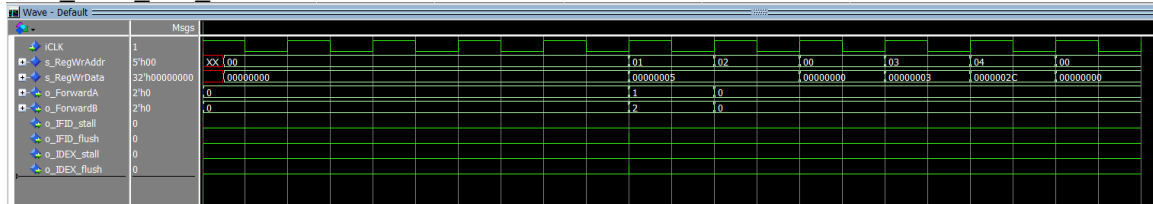
StoreDataFwd:



Again, no stalls or flushes are needed, but a forward is used.

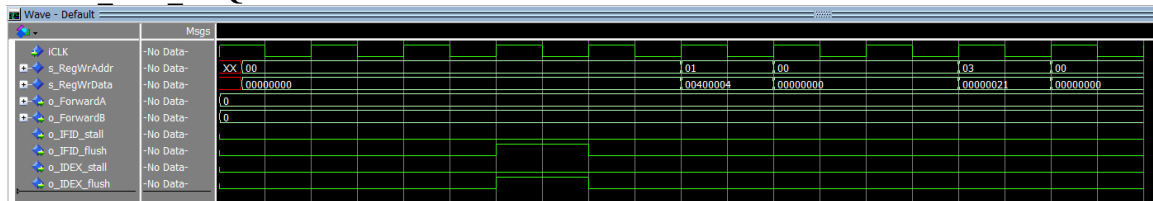
Combo1:

Test BNE_Not_Take:



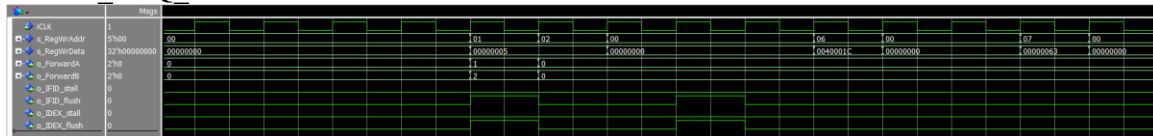
Branch occurs in the execute stage. If it doesn't branch, no flushes are needed as the sequential instructions are the ones that need to be run.

Combo_JAL_BEQ:



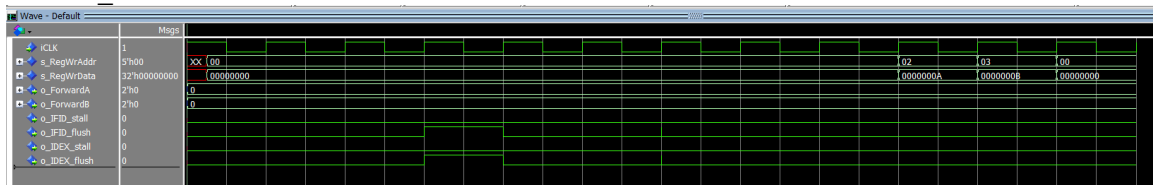
Jump and link causes the flushing, branch not taken so no flush.

Combo_BEQ_JAL:



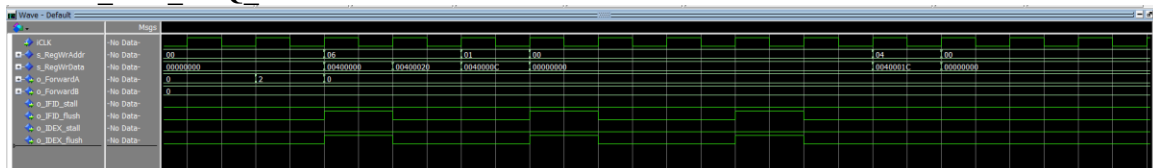
Branch is taken so it flushes, JAL flushes as always.

Combo_2Branches:



One branch is taken and one is not. Only the one that is taken flushes.

Combo_JAL_BEQ_JALR:



All 3 instructions cause the PC to change, and therefore, the instructions before them are all flushed.

[2.e.i] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

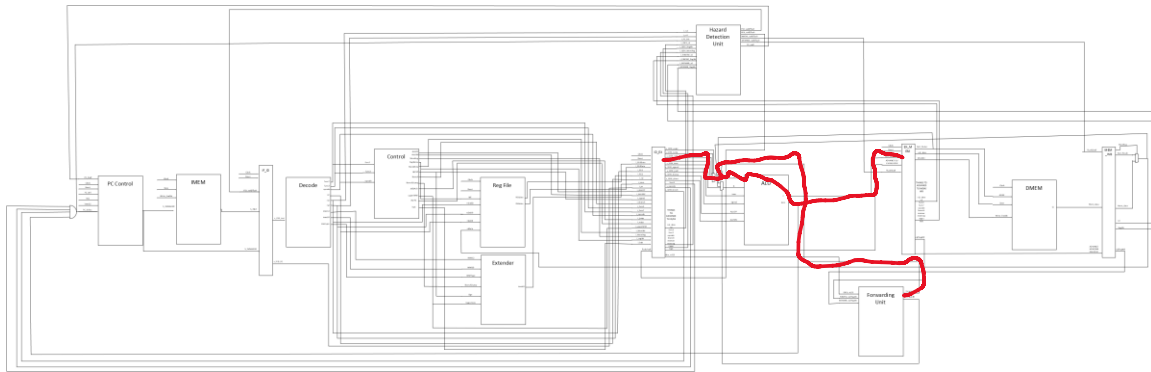
Program Name	Forwarding Path	Stall Required	Signals Exercised
ComputeUse_EXMEM	EX/MEM→EX	No	s_ALUResult, s_EXMEM_ALU_result, s_ALU_Op1, s_ALU_Op2
ComputeUse_MEMWB	MEM/WB→EX	No	s_MEMWB_ALU_result
LoadUse_1	MEM/WB→EX	Yes	s_DMemOut, s_LoadData
LoadUse_Double	MEM/WB→EX	Yes	s_LoadData
WBUse	WB→ID	No	s_RegWrData
StoreAddrFwd	EX/MEM→MEM	No	s_DMemAddr
StoreDataFwd	EX/MEM→MEM	No	s_DMemData
Combo1	EX/MEM→EX + MEM/WB→EX	Yes	all forwarding signals
Combo2	MEM/WB→EX, EX/MEM→EX	Yes	ALU+Load paths
Combo3_Full	ALL paths	Yes	Complete datapath

[2.e.ii] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Program Name	Instruction(s) Tested	Stage PC Changes	Flush Expected	Stalls Needed	Pipeline Stages Flushed
Test_JAL	JAL	EX	Yes	No	2 stages
Test_JALR	JALR	EX	Yes	No	2 stages
Test_BEQ	BEQ (taken)	EX	Yes	No	2 stages
Test_BNE_Not_Take	BNE (not taken)	N/A	No	No	None
Combo_JAL_BEQ	JAL then BEQ	EX	Yes	No	2 stages
Combo_BEQ_JAL	BEQ then JAL	EX	Yes	No	4 stages
Combo_2Branches	BEQ + BNE	EX	Yes	No	2 stages
Combo_JAL_BEQ_JALR	JAL → BEQ → JALR	EX	Yes	No	6 stages

[2.f] report the maximum frequency your hardware-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

F_{max} = 42.23 MHz



ID_EX -> Forwarding Unit -> ALU -> EX_MEM